

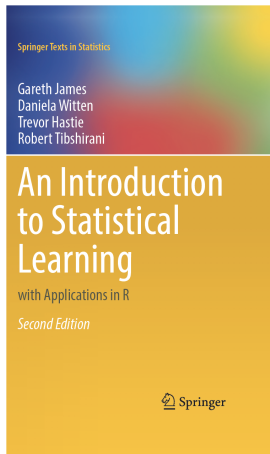
ML101
Chapter 09: Support Vector Machines
An Introduction to Statistical Learning, ISLRv2

James, Witten, Hastie & Tibshirani

October 1, 2026

An Introduction to Statistical Learning with Applications in R

a.k.a. ISLR



Content has been extracted from *An Introduction to Statistical Learning with Applications in R*, Second Edition, by James, Witten, Hastie & Tibshirani. Springer. 2023.
Visit <https://www.statlearning.com/>.

Support Vector Machines

Here we approach the two-class classification problem in a direct way:
We try and find a plane that separates the classes in feature space.

If we cannot, we get creative in two ways:

- ▶ We soften what we mean by “separates”, and
- ▶ We enrich and enlarge the feature space so that separation is possible.

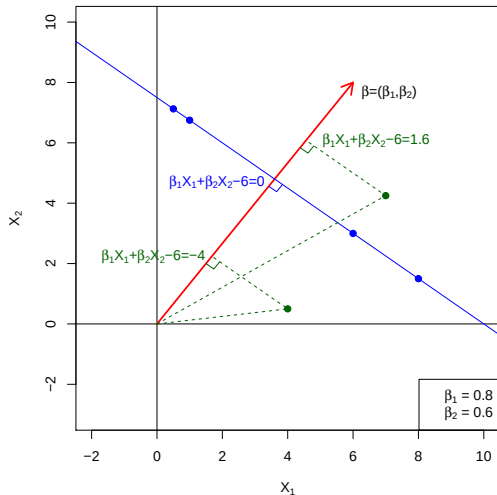
What is a Hyperplane?

- ▶ A hyperplane in p dimensions is a flat affine subspace of dimension $p - 1$.
- ▶ In general the equation for a hyperplane has the form

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p = 0$$

- ▶ In $p = 2$ dimensions a hyperplane is a line.
- ▶ If $\beta_0 = 0$, the hyperplane goes through the origin, otherwise not.
- ▶ The vector $\beta = (\beta_1, \beta_2, \dots, \beta_p)$ is called the normal vector - it points in a direction orthogonal to the surface of a hyperplane.

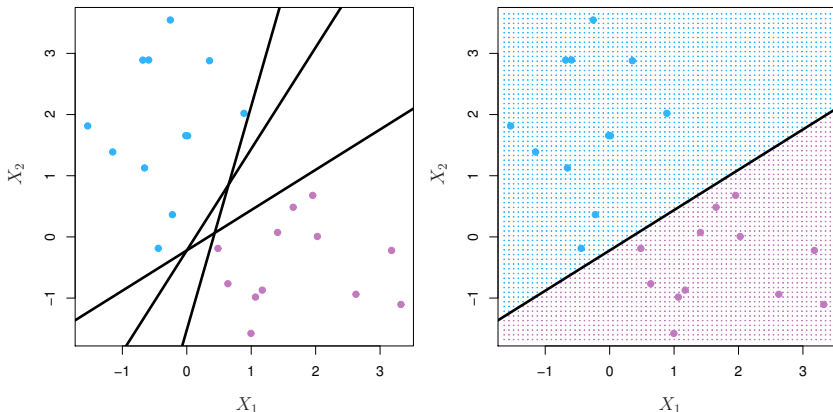
Hyperplane in 2 Dimensions



$$\begin{aligned}\beta_1 X_1 + \beta_2 X_2 - 6 &= 0 \\ \beta_1 X_1 + \beta_2 X_2 - 6 &= -4 \\ \beta_1 X_1 + \beta_2 X_2 - 6 &= 1.6 \\ \beta &= (\beta_1, \beta_2), \text{ with } \beta_1 = 0.8, \beta_2 = 0.6\end{aligned}$$

A 2D plot illustrating a separating hyperplane (blue line) with normal vector β (red arrow) and equations for boundaries.

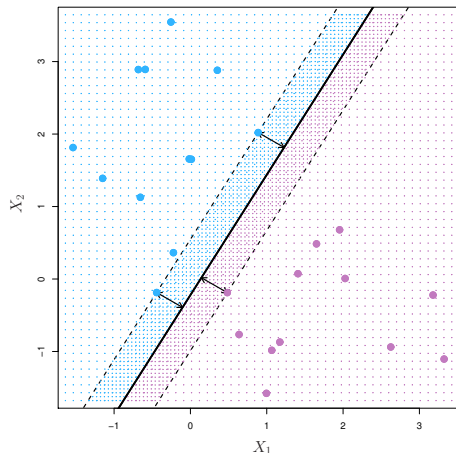
Separating Hyperplanes



Linearly separable data with multiple potential separating hyperplanes. The margin of classification shaded in blue and pink.

- ▶ If $f(X) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$ then $f(X) > 0$ for points on one side of the hyperplane, and $f(X) < 0$ for points on the other.
- ▶ If we code the colored points as $Y_i = +1$ for blue, say, and $Y_i = -1$ for mauve, then if $Y_i \cdot f(X_i) > 0$ for all i , $f(X) = 0$ defines a separating hyperplane.

Maximal Margin Classifier



A 2D plot showing the maximal margin classifier. The margin is maximized between the two classes with arrows pointing to the support vectors.

Among all separating hyperplanes, find the one that makes the biggest gap or margin between the two classes.

Constrained optimization problem:

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{maximize}} \quad M$$

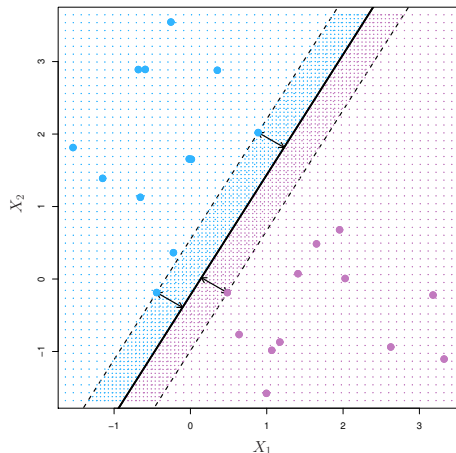
subject to

$$\sum_{j=1}^p \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M$$

for all $i = 1, \dots, N$

Maximal Margin Classifier



A 2D plot showing the maximal margin classifier. The margin is maximized between the two classes with arrows pointing to the support vectors.

Among all separating hyperplanes, find the one that makes the biggest gap or margin between the two classes.

Constrained optimization problem:

$$\underset{\beta_0, \beta_1, \dots, \beta_p}{\text{maximize}} \quad M$$

subject to

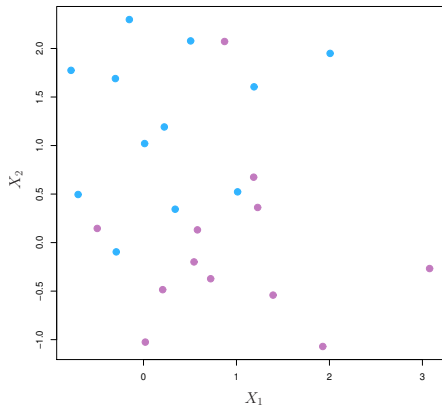
$$\sum_{j=1}^p \beta_j^2 = 1,$$

$$y_i(\beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}) \geq M$$

for all $i = 1, \dots, N$

This can be rephrased as a convex quadratic program, and solved efficiently. The function `svm()` in package `e1071` solves this problem efficiently.

Non-separable Data

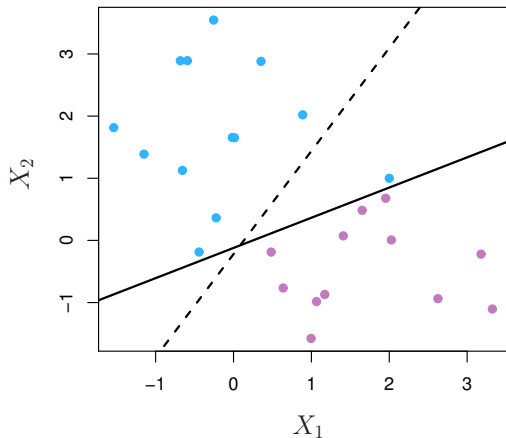
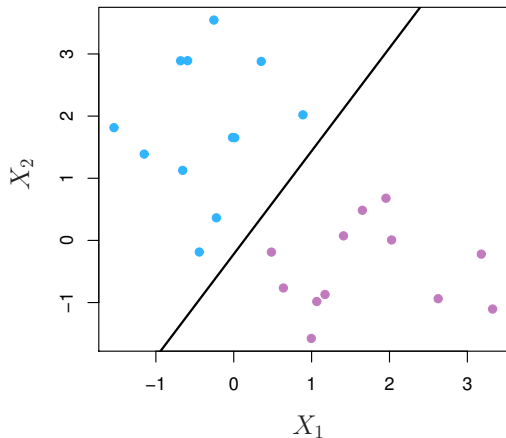


The data on the left are not separable by a linear boundary.

This is often the case, unless $N < p$.

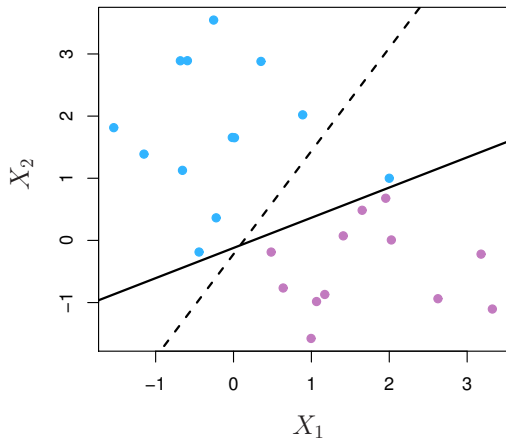
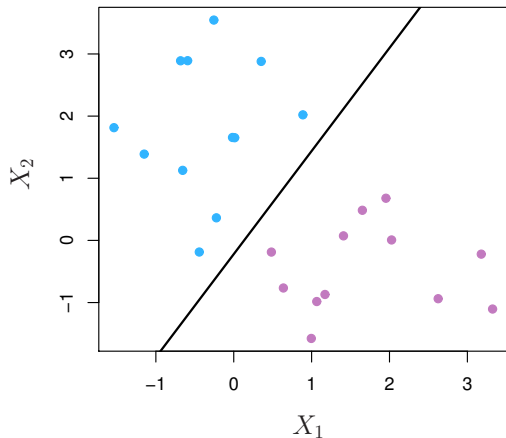
A scatterplot showing non-separable classes where blue and mauve points are intermixed.

Noisy Data



Sometimes the data are separable, but noisy. This can lead to a poor solution for the maximal-margin classifier.

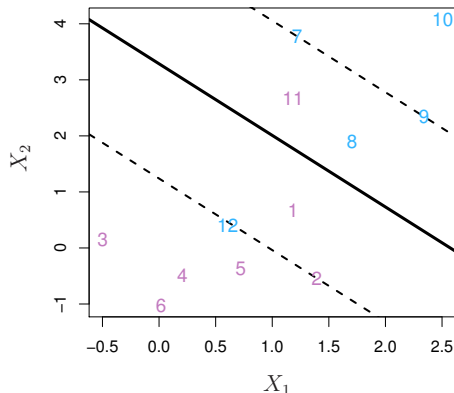
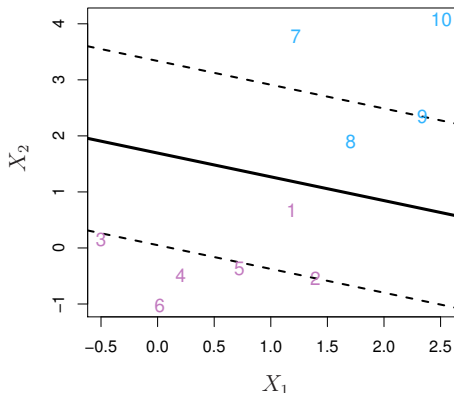
Noisy Data



Sometimes the data are separable, but noisy. This can lead to a poor solution for the maximal-margin classifier.

The support vector classifier maximizes a soft margin.

Support Vector Classifier



Plots demonstrating a soft margin classifier. Numbers identifies support vectors, some of which fall within the the wrong side.

$$\begin{aligned} & \underset{\beta_0, \beta_1, \dots, \beta_p, \epsilon_1, \dots, \epsilon_n}{\text{maximize}} && M && \text{subject to } \sum_{j=1}^p \beta_j^2 = 1, \\ & y_i(\beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \dots + \beta_p x_{ip}) \geq M(1 - \epsilon_i), \\ & \epsilon_i \geq 0, && \sum_{i=1}^n \epsilon_i \leq C \end{aligned}$$

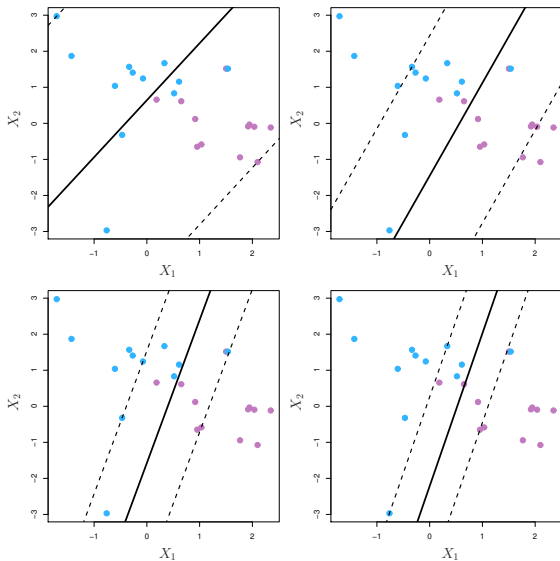
Support Vector Classifier –continued

- ▶ **The Problem:** Real-world data is often noisy or non-separable. A strict maximal margin classifier is highly sensitive to outliers and can overfit.
- ▶ **Soft Margin Solution:** We allow some observations to violate the margin, or even fall on the wrong side of the separating hyperplane.
- ▶ **Slack Variables (ϵ_i):** The ϵ_i values are “slack variables” that measure how much an individual data point is allowed to violate its margin:
 - ▶ $\epsilon_i = 0$: Point is on the correct side of the margin.
 - ▶ $\epsilon_i > 0$: Point has violated the margin and sits on the wrong side of it.
 - ▶ $\epsilon_i > 1$: Point has violated the margin so severely that it is located on the wrong side of the separating hyperplane itself (a misclassification).

Support Vector Classifier –continued

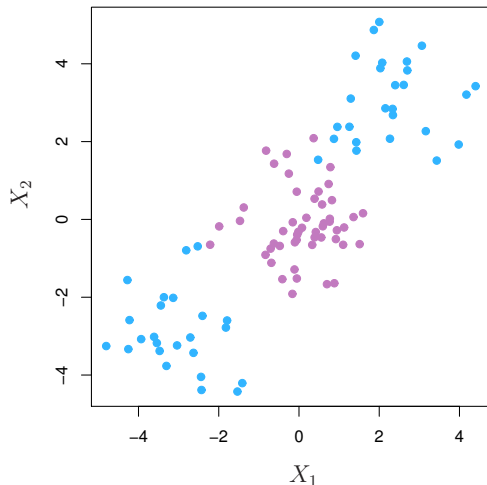
- ▶ **The Problem:** Real-world data is often noisy or non-separable. A strict maximal margin classifier is highly sensitive to outliers and can overfit.
- ▶ **Soft Margin Solution:** We allow some observations to violate the margin, or even fall on the wrong side of the separating hyperplane.
- ▶ **Regularization Budget (C):** The tuning parameter C bounds the total sum of margin violations ($\sum \epsilon_i \leq C$):
 - ▶ C acts as a regularization parameter that controls the bias-variance trade-off of the classifier.
 - ▶ A larger C provides a higher tolerance for overlap and margin violations, resulting in a wider margin that is influenced by more points (yielding lower variance but potentially higher bias).
 - ▶ A smaller C strictly limits the allowed slack, forcing a narrower margin that fits the training data more rigidly (yielding lower bias but higher variance).

Support Vector Classifier



The effect of varying the regularization parameter C on the soft margin width and decision boundary.

Linear boundary can fail



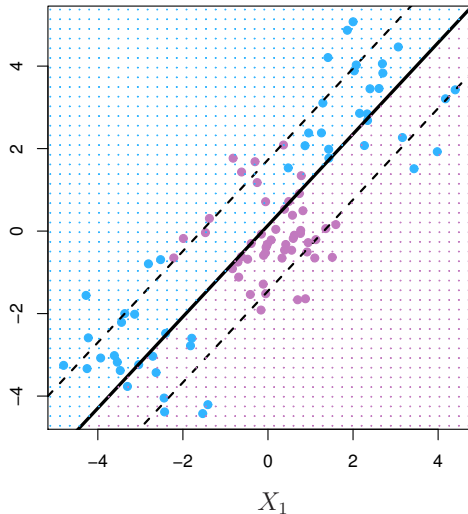
One class concentrated in the center and the other distributed around it.

Sometime a linear boundary simply won't work, no matter what value of C .

The example on the left is such a case.

What to do?

Linear boundary can fail



A linear boundary fails to separate them.

Sometime a linear boundary simply won't work, no matter what value of C .

The example on the left is such a case.

What to do?

Feature Expansion

- ▶ Enlarge the space of features by including transformations; e.g. $X_1^2, X_1^3, X_1X_2, X_1X_2^2, \dots$. Hence go from a p -dimensional space to a $M > p$ dimensional space.
- ▶ Fit a support-vector classifier in the enlarged space.
- ▶ This results in non-linear decision boundaries in the original space.

Feature Expansion

- ▶ Enlarge the space of features by including transformations; e.g. $X_1^2, X_1^3, X_1X_2, X_1X_2^2, \dots$. Hence go from a p -dimensional space to a $M > p$ dimensional space.
- ▶ Fit a support-vector classifier in the enlarged space.
- ▶ This results in non-linear decision boundaries in the original space.

Example: Suppose we use $(X_1, X_2, X_1^2, X_2^2, X_1X_2)$ instead of just (X_1, X_2) . Then the decision boundary would be of the form

$$\beta_0 + \beta_1X_1 + \beta_2X_2 + \beta_3X_1^2 + \beta_4X_2^2 + \beta_5X_1X_2 = 0$$

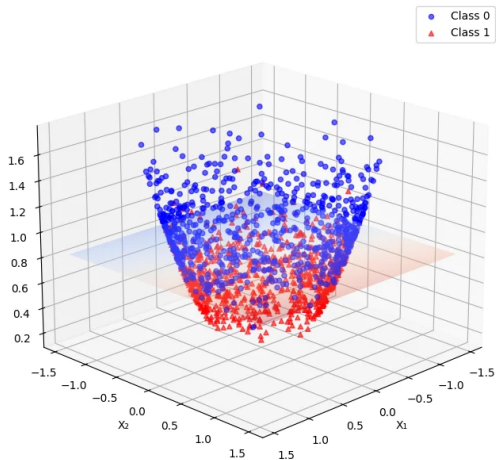
This leads to nonlinear decision boundaries in the original space (quadratic conic sections).

Feature Expansion –continued

Polynomial Kernel Transformation:
 $\Phi(x_1, x_2) \rightarrow (x_1, x_2, x_1^2 + x_2^2)$

Classes are linearly separable
in transformed space

3D Transformed Dataset with Linear Separator



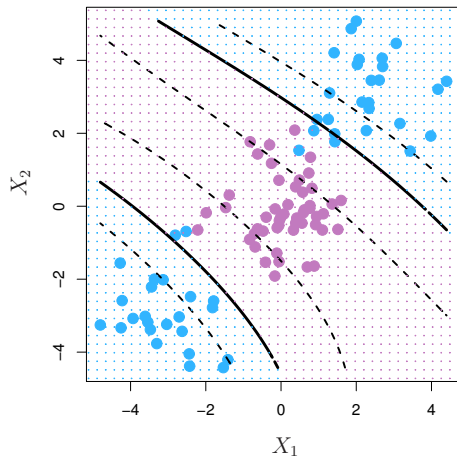
<https://www.freecodecamp.org/news/svm-kernels-how-to-tackle-nonlinear-data-in-machine-learning/>

Cubic Polynomials

Here we use a basis expansion of cubic polynomials.

From 2 variables to 9.

The support-vector classifier in the enlarged space solves the problem in the lower-dimensional space.



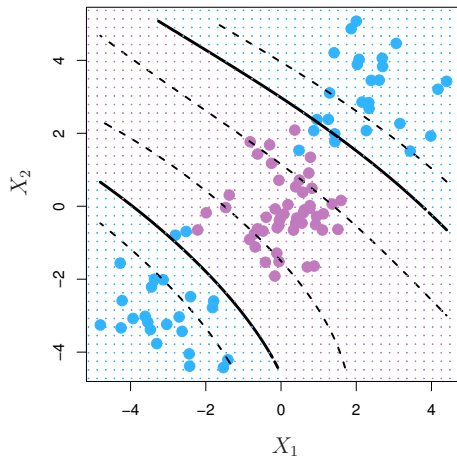
Decision boundary of a Support Vector Classifier using a cubic polynomial kernel, successfully separating the non-linear classes.

Cubic Polynomials

Here we use a basis expansion of cubic polynomials.

From 2 variables to 9.

The support-vector classifier in the enlarged space solves the problem in the lower-dimensional space.



Decision boundary of a Support Vector Classifier using a cubic polynomial kernel, successfully separating the non-linear classes.

$$\beta_0 + \beta_1 X_1 + \beta_2 X_2 + \beta_3 X_1^2 + \beta_4 X_2^2 + \beta_5 X_1 X_2 + \beta_6 X_1^3 + \beta_7 X_2^3 + \beta_8 X_1 X_2^2 + \beta_9 X_1^2 X_2 = 0$$

Nonlinearities and Kernels

- ▶ Polynomials (especially high-dimensional ones) get wild rather fast.
- ▶ There is a more elegant and controlled way to introduce nonlinearities in support-vector classifiers — through the use of **kernels**.
- ▶ Before we discuss these, we must understand the role of **inner products** in support-vector classifiers.

The Inner Product

- ▶ The equation $\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij}x_{i'j}$ calculates the inner product between two distinct observation vectors.
- ▶ This mathematical operation multiplies the corresponding individual components of the two vectors and sums the cross products together.
- ▶ In the context of support vector machines, this inner product functions as an abstract measure of “similarity” between those two observations.

The Inner Product

- ▶ The equation $\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij}x_{i'j}$ calculates the inner product between two distinct observation vectors.
- ▶ This mathematical operation multiplies the corresponding individual components of the two vectors and sums the cross products together.
- ▶ In the context of support vector machines, this inner product functions as an abstract measure of “similarity” between those two observations.
 - ▶ When two observations share large feature values in the same direction, multiplying them together yields large positive numbers.
 - ▶ Summing these cross-products results in a large total inner product, indicating a strong similarity between the two data points.

The Classifier Function

- ▶ The equation $f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$ shows that the classifier can be written entirely in terms of these inner products.
- ▶ Instead of using a parameter for each original feature, the model assigns a specific weight parameter (α_i) to every single training observation (n parameters total).
- ▶ To evaluate a new point (x), the model computes its inner product (similarity) with every existing training point (x_i) and multiplies it by that point's weight.

The Classifier Function

- ▶ The equation $f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$ shows that the classifier can be written entirely in terms of these inner products.
- ▶ Instead of using a parameter for each original feature, the model assigns a specific weight parameter (α_i) to every single training observation (n parameters total).
- ▶ To evaluate a new point (x), the model computes its inner product (similarity) with every existing training point (x_i) and multiplies it by that point's weight.
 - ▶ The inner product directly quantifies the relationship between observations by functioning as a measure of similarity.
 - ▶ This mathematical property allows a linear support vector machine to evaluate new data without applying rigid formulas to isolated feature coordinates.
 - ▶ The model classifies a new point by calculating its pairwise similarity exclusively against the specific, borderline training observations, which are known as support vectors.

The Support Set

- ▶ The formula $f(x) = \beta_0 + \sum_{i \in S} \hat{\alpha}_i \langle x, x_i \rangle$ reveals the efficiency of this method.
- ▶ When the model is fitted, many of the α_i weights end up being exactly zero.
- ▶ The specific training observations that retain a non-zero weight ($\alpha_i > 0$) form the “support set” ($\mathcal{S}$) and are known as the support vectors.
- ▶ Because most points have a weight of zero, evaluating a new observation only requires computing its similarity to this small handful of support vectors, effectively ignoring the rest of the dataset for the final calculation.

Inner products and support vectors –summary

- ▶ Inner product between vectors...

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}$$

Inner products and support vectors –summary

- ▶ Inner product between vectors...

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}$$

- ▶ The linear support vector classifier, with n **parameters**, can be represented as...

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$$

Inner products and support vectors –summary

- ▶ Inner product between vectors...

$$\langle x_i, x_{i'} \rangle = \sum_{j=1}^p x_{ij} x_{i'j}$$

- ▶ The linear support vector classifier, with n **parameters**, can be represented as...

$$f(x) = \beta_0 + \sum_{i=1}^n \alpha_i \langle x, x_i \rangle$$

- ▶ To estimate the parameters $\alpha_1, \dots, \alpha_n$ and β_0 , all we need are the $\binom{n}{2}$ inner products $\langle x_i, x_{i'} \rangle$ between all pairs of training observations.

It turns out that most of the $\hat{\alpha}_i$ can be zero:

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i \langle x, x_i \rangle$$

$\mathcal{S}$ is the **support set** of indices i such that $\hat{\alpha}_i > 0$.

Kernels and Support Vector Machines

- If we can compute inner-products between observations, we can fit a SV classifier. Can be quite abstract!

Kernels and Support Vector Machines

- ▶ If we can compute inner-products between observations, we can fit a SV classifier. Can be quite abstract!
- ▶ Some special (*polynomial*) **kernel functions** can do this for us. E.g.

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d$$

computes the inner-products needed for d dimensional polynomials - $\binom{p+d}{d}$ basis functions!

Try it for $p = 2$ and $d = 2$.

Kernels and Support Vector Machines

- ▶ If we can compute inner-products between observations, we can fit a SV classifier. Can be quite abstract!
- ▶ Some special (*polynomial*) **kernel functions** can do this for us. E.g.

$$K(x_i, x_{i'}) = \left(1 + \sum_{j=1}^p x_{ij} x_{i'j} \right)^d$$

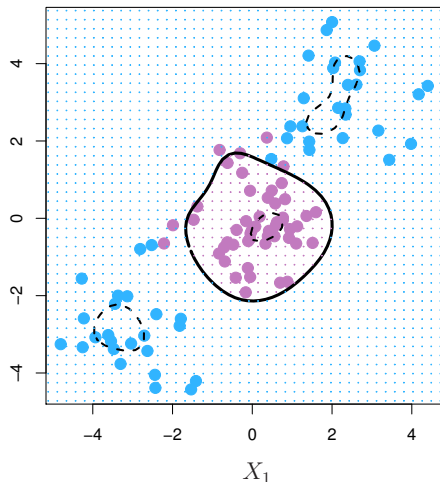
computes the inner-products needed for d dimensional polynomials - $\binom{p+d}{d}$ basis functions!

Try it for $p = 2$ and $d = 2$.

- ▶ The solution has the form

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i).$$

Radial Kernel



Radial kernel cleanly encapsulating non-linear clusters.

$$K(x_i, x_{i'}) = \exp \left(-\gamma \sum_{j=1}^p (x_{ij} - x_{i'j})^2 \right)$$

$$f(x) = \beta_0 + \sum_{i \in \mathcal{S}} \hat{\alpha}_i K(x, x_i)$$

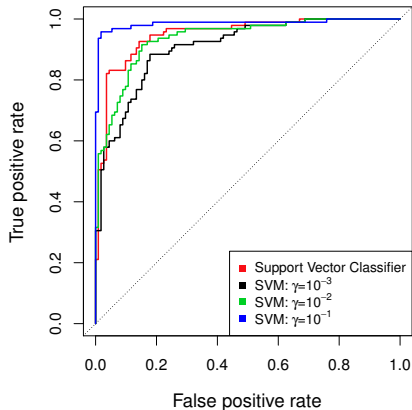
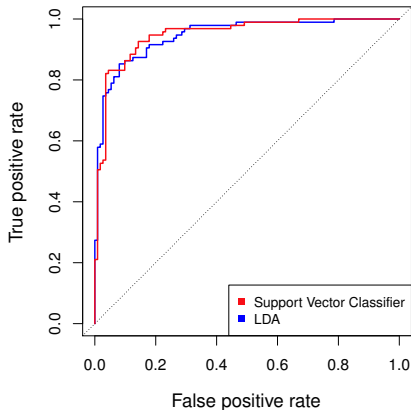
The calculation mirrors the core of a multivariate Gaussian distribution.

Dimensionality “Squashing”: Controls variance by squashing down most dimensions severely.

Local Behavior: Distant training observations have virtually no impact on predicting a specific test point

γ parameter: a positive tuning parameter that controls the flexibility of the boundary.

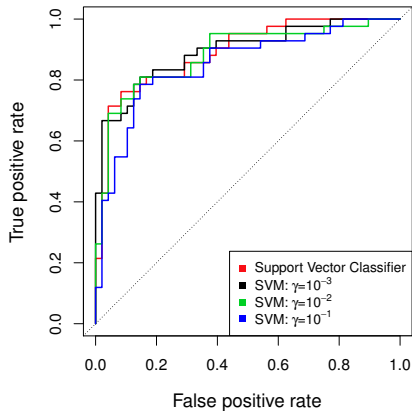
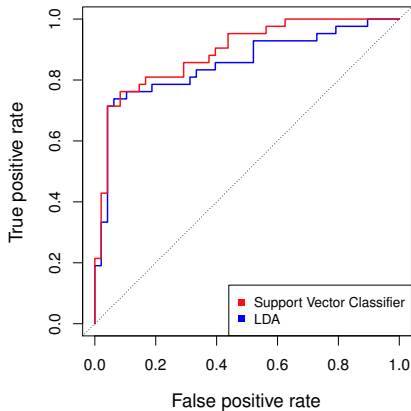
Example: Heart Data



ROC curves on training data. LDA vs Support Vector Classifier (left). SVMs with different γ values (right).

ROC curve is obtained by changing the threshold 0 to threshold t in $\hat{f}(X) > t$ and recording **false positive** and **true positive** rates as t varies. Here we see ROC curves on training data.

Example continued: Heart Test Data



ROC curves on test data. LDA and Support Vector Classifier. SVMs with different γ values (10^{-3} , 10^{-2} , 10^{-1}).

SVMs: more than 2 classes?

- ▶ The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?

SVMs: more than 2 classes?

- ▶ The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?
 - ▶ **OVA** One versus All. Fit K different 2-class SVM classifiers $\hat{f}_k(x)$, $k = 1, \dots, K$; each class versus the rest. Classify x^* to the class for which $\hat{f}_k(x^*)$ is largest.

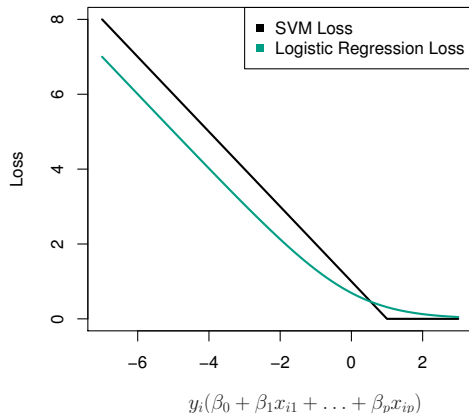
SVMs: more than 2 classes?

- ▶ The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?
 - ▶ **OVA** One versus All. Fit K different 2-class SVM classifiers $\hat{f}_k(x)$, $k = 1, \dots, K$; each class versus the rest. Classify x^* to the class for which $\hat{f}_k(x^*)$ is largest.
 - ▶ **OVO** One versus One. Fit all $\binom{K}{2}$ pairwise classifiers $\hat{f}_{kl}(x)$. Classify x^* to the class that wins the most pairwise competitions.

SVMs: more than 2 classes?

- ▶ The SVM as defined works for $K = 2$ classes. What do we do if we have $K > 2$ classes?
 - ▶ **OVA** One versus All. Fit K different 2-class SVM classifiers $\hat{f}_k(x)$, $k = 1, \dots, K$; each class versus the rest. Classify x^* to the class for which $\hat{f}_k(x^*)$ is largest.
 - ▶ **OVO** One versus One. Fit all $\binom{K}{2}$ pairwise classifiers $\hat{f}_{kl}(x)$. Classify x^* to the class that wins the most pairwise competitions.
- ▶ Which to choose? If K is not too large, use OVO.

Support Vector versus Logistic Regression?



Line plot comparing Hinge Loss (SVM) and Negative Log-Likelihood Loss (Logistic Regression). Both look very similar.

With $f(X) = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p$ can rephrase support-vector classifier optimization as

$$\min_{\beta_0, \beta_1, \dots, \beta_p} \left\{ \sum_{i=1}^n \max[0, 1 - y_i f(x_i)] + \lambda \sum_{j=1}^p \beta_j^2 \right\}$$

This has the form **loss plus penalty**.

The loss is known as the **hinge loss**.

Very similar to “loss” in logistic regression (negative log-likelihood).

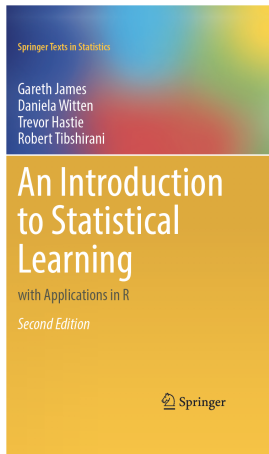
Which to use: SVM or Logistic Regression

- ▶ When classes are (nearly) separable, SVM does better than LR. So does LDA.
- ▶ When not, LR (with ridge penalty) and SVM very similar.
- ▶ If you wish to estimate probabilities, LR is the choice.
- ▶ For nonlinear boundaries, kernel SVMs are popular. Can use kernels with LR and LDA as well, but computations are more expensive.

End of Chapter 9.

An Introduction to Statistical Learning with Applications in R

a.k.a. ISLR



Content has been extracted from *An Introduction to Statistical Learning with Applications in R*, Second Edition, by James, Witten, Hastie & Tibshirani. Springer. 2023.
Visit <https://www.statlearning.com/>.